

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO
SCC0504 - Programação Orientada a Objetos

ESPECIFICAÇÃO DE REQUISITOS E ARQUITETURA DE SOFTWARE
JAVA EVENT PLANNER

Discentes:

Mateus Vargas Saracuze – 13674087

Nícolas Silva Scorza – 17025079

Rafael Mendonça Duarte – 168817608

Docente:

Dilvan de Abreu Moreira



São Carlos - SP

2026

SUMÁRIO

1. VISÃO GERAL DO PRODUTO E INSTRUÇÕES DE EXECUÇÃO

2. REQUISITOS FUNCIONAIS (RFS)

3. REQUISITOS NÃO FUNCIONAIS E RESTRIÇÕES ARQUITETURAIS

4. DECISÕES ARQUITETURAIS CONSOLIDADAS (ADRS)

5. PRINCÍPIOS DE ORIENTAÇÃO A OBJETOS APLICADOS

6. DIAGRAMA TÉCNICO DO PROGRAMA

1. Visão Geral do Produto e Instruções de Execução

O Java Event Planner é um aplicativo desktop offline de gerenciamento de calendário e agendamento de equipe desenvolvido em Java Swing. O sistema permite que múltiplos usuários pré-definidos gerenciem seus próprios eventos e visualizem a disponibilidade de colegas através de uma interface de planner dinâmico.

Como Executar a Aplicação:

```
javac Main.java  
java Main
```

Após a inicialização, a aplicação carregará o estado padrão com baseado no usuário ativo configurado na sessão do protótipo, lendo e inicializando as estruturas diretamente dos arquivos locais events.csv e reminders.csv.

2. Requisitos Funcionais (RFs)

A tabela abaixo apresenta a listagem atualizada de todos os requisitos funcionais implementados no sistema, refletindo a consolidação do MVP e os mecanismos de segurança e validação em série.

ID	Nome	Descrição	Prioridade
RF01	Autenticação por Seleção	Identificação do usuário ativo no startup da sessão para controle contextual de permissões (pré-definido).	MUST
RF02	Ownership de Evento	Todo novo evento registra automaticamente o usuário logado como seu dono legítimo no campo específico do modelo.	MUST
RF03	Controle de Acesso	Bloqueio dinâmico na interface: os botões de edição e exclusão de eventos são desabilitados se o dono do registro for diferente do usuário ativo da sessão.	MUST
RF09	Criar Evento	Formulário para inclusão de novos eventos contendo validações de preenchimento (título não vazio, data ISO válida, hora válida, duração positiva e categoria).	MUST
RF10	Editar Evento	Permite ao dono alterar os dados do evento	MUST

ID	Nome	Descrição	Prioridade
		em memória. O formulário reaproveita a estrutura injetando o modelo existente e salvando via reescrita completa.	
RF11	Excluir Evento	Permite ao dono remover um evento após confirmação. A remoção limpa em cascata os lembretes associados e atualiza o arquivo físico.	MUST
RF12	Busca por Palavra-chave	Interface baseada em JList que realiza filtragem em tempo real (título, descrição e local) à medida que o texto é digitado, exibindo botões de ação contextuais ao dono.	MUST
RF17-19	Configuração de Recorrência	Inclusão de JCheckBox "Repetir evento". Quando marcado, habilita a seleção de frequência (Diário, Semanal, Mensal) e número de repetições (limite estrito de 2 a 50).	MUST
RF20-21	Geração de Série e Alertas	Pré-cálculo síncrono das datas futuras no ato do salvamento, gerando registros planos independentes que compartilham o mesmo recurrence_id, replicando lembretes.	MUST
RF22	Indicador Visual de Série	Eventos que pertencem a uma série recorrente exibem automaticamente o caractere identificador ☺ ao lado de sua representação textual.	SHOULD
RF23-24	Gerenciamento de Escopo da Série	Ao editar ou excluir uma ocorrência vinculada, um diálogo modal interroga se o escopo deve afetar "Somente esta" ou "Esta e as futuras", preservando o histórico passado.	MUST
RF25	Prevenção de	Mecanismo anti-colisão que realiza um Dry Run na coleção viva em memória,	MUST

ID	Nome	Descrição	Prioridade
	Conflitos	impedindo o salvamento se houver sobreposição de horários na agenda do usuário.	

3. Requisitos Não Funcionais e Restrições Arquiteturais

- **RNF01 (Single-Threaded UI):** Todo o processamento visual e lógico é executado na Event Dispatch Thread (EDT) nativa do Swing.
- **RNF02 (In-Memory Processing):** Processamento feito majoritariamente em memória.
- **RNF03 (Persistência Híbrida):** Operações de inserção pura utilizam a estratégia de anexo no fim do arquivo (Append). Operações de modificação ou exclusão disparam uma reescrita completa (Full Rewrite) do arquivo CSV correspondente com base no estado validado da memória.
- **RNF05 (Tratamento de Erros Seguro):** Exceções de I/O ou falhas de validação de dados de entrada são interceptadas por blocos try/catch e reportadas amigavelmente via JOptionPane. Stack traces do sistema nunca são expostos ao usuário.

4. Decisões Arquiteturais Consolidadas (ADRs)

ID	Decisão Arquitetural	Justificativa Prática
DT01	Processamento majoritariamente em Memória	Diminui os gargalos físicos de leitura e escrita em disco durante interações frequentes com a UI, mantendo a responsividade do calendário mensal.
DT03	Persistência via Full Rewrite	Contorna as limitações de manipulação sequencial de arquivos texto planos (.csv), permitindo atualizar ou deletar registros com segurança absoluta em nível atômico.
DT07	Recorrência Plana em Linhas Independentes	Cada ocorrência é salva como um evento autônomo, simplificando as consultas e respeitando o modelo relacional simples.

ID	Decisão Arquitetural	Justificativa Prática
DT08	Edição e Exclusão Baseada em recurrence_id	Permite a propagação de alterações e remoções em massa para eventos futuros ("Esta e as futuras") utilizando filtragem condicional simples de ID combinado com data de referência.
DT09	Validação de Interseção de Tempo Síncrona	Garante a integridade da agenda do usuário calculando a sobreposição de horários (InícioCompromisso < FimExistente E FimCompromisso > InícioExistente) diretamente na EDT antes de efetivar o salvamento.

5. Princípios de Orientação a Objetos Aplicados

O design de software foi planejado seguindo boas práticas de Engenharia de Software e Orientação a Objetos:

Conceito	Onde no código	Por que foi aplicado
Encapsulamento	Event, Reminder (campos <i>private</i> + getters/setters); EventDao/ReminderDao	Blindam o estado interno dos modelos e ocultam o formato do CSV do resto da aplicação, expondo apenas operações controladas de negócio e persistência.
Herança (reuso)	EventListView → DailyView, WeeklyView AbstractCsvDao → EventDao, ReminderDao BaseFormDialog → EventForm	A superclasse visual centraliza o painel rolável e métodos protected; as subclasses só implementam a filtragem. O AbstractCsvDao reaproveita a lógica de manipulação atômica de arquivos (Full Rewrite) para qualquer entidade. O BaseFormDialog aplica o padrão de projeto Template Method, padronizando o ciclo de vida (carregar, validar, salvar) onde as subclasses só definem campos locais.
Herança (framework)	MainWindow : JFrame; diálogos : JDialog; TeamScheduleTableModel :	Estende classes nativas do Swing para herdar comportamento de janela e o contrato MVC da JTable.

Conceito	Onde no código	Por que foi aplicado
	AbstractTableModel; TeamScheduleCellRenderer : DefaultTableCellRenderer	
Polimorfismo	Sobrescrita de getValueAt, getTableCellRendererComponent , getListCellRendererComponent; interface própria DaySelectionListener Interface RecurrenceStrategy (DailyRecurrence, etc.); Polimorfismo nos DAOs na conversão de linha CSV.	A JTable opera sobre o contrato abstrato sem conhecer a regra de negócio; a interface Listener funciona como um Observer simplificado. A RecurrenceStrategy elimina estruturas condicionais (if/switch) complexas ao injetar a regra de cálculo de datas futuras. O DAO genérico processa qualquer classe delegando a serialização específica para a subclasse.
Composição / Associação	MainWindow compõe as views; AppState agrega os dois DAOs; injeção da mesma instância de EventDao	Uma edição em qualquer ponto reflete em todas as visões sem reler o disco, pois compartilham a coleção viva em memória.
Separação de responsabilidades	Pacotes model / persistence / service / ui	Isola dados, I/O, regras de sessão e apresentação, atendendo ao critério de <i>good separation of concerns</i> da rubrica.

6. Diagrama técnico geral do programa

O diagrama apresenta a arquitetura completa da aplicação em quatro camadas, destacando a herança de framework (Swing) e a herança interna de reuso, além da composição entre MainWindow e suas visões. Mostra ainda a persistência via DAOs com listas em memória, o estado de sessão em AppState e o modelo de domínio (Event, Reminder, Category).

